

TECHNICAL REQUIREMENTS DOCUMENT

Meridian

Personal AI Investment Agent -
production-grade blueprint covering the
Next.js 16 frontend, Java 21 + Spring Boot
backend, alert rule engine, Spring AI agent,
the Indian market-data provider strategy,
and the phased delivery plan.

Version 1.0 | Next.js 16 | React 19 | Java 21 +
Spring Boot 3.5 | PostgreSQL 16 | Redis 7 | Spring
AI

Table of Contents

1. Document Control

3

2. System Overview and Architecture

3

2.1 Product context

3

2.2 Architecture at a glance

3

2.3 Architecture principles

4

3. Technology Stack and Key Decisions

4

4. Market Data Research and Provider Strategy

5

4.1 Provider landscape

5

4.2 News and IPO data

6

4.3 Recommended strategy

7

5. Backend Architecture (Spring Boot)

7

5.1 Module structure

7

5.2 Cross-cutting components

8

6. Alert Rule Engine Design

8

6.1 Rule model

9

6.2 Evaluation pipeline

9

6.3 Background job cadences

9

7. AI Agent Architecture (Spring AI)

10

7.1 Orchestration flow

10

7.2 Tool registry (initial set)

10

7.3 Guardrails and audit

11

8. Data Model and Persistence

11

9. API Contract

12

9.1 Conventions 12

9.2 Endpoint inventory 13

9.3 Example payloads 13

10. Security and Compliance 14

11. Frontend Architecture 14

11.1 Current module map (delivered) 14

11.2 Backend integration strategy (this release) 15

11.3 Screen inventory and status 15

12. Non-Functional Requirements 16

13. Environments and DevOps 17

14. Delivery Plan and Current Status 17

15. Risks and Mitigations 18

Appendix A. Agent System Prompt (baseline) 19

Appendix B. References 19

1. Document Control

Field	Value
Product	Meridian - Personal AI Investment Agent (PA)
Document type	Technical Requirements Document (TRD)
Source document	Personal AI Investment Agent PRD v1.0 (September 2026)
Audience	Founder / developer / designer
Current frontend	Next.js 16 + React 19 + TypeScript (strict) + Tailwind CSS 4 + Recharts
Planned backend	Java 21 + Spring Boot 3.5.x, PostgreSQL 16, Redis 7, Spring AI 1.x
Financial safety posture	Decision support first; no autonomous order placement in MVP
Status	Frontend foundation delivered; backend build starts after this document

Table 1-1: Document control

This TRD translates the approved PRD into an implementable technical blueprint. It fixes the system architecture, the backend technology decisions (Spring Boot), the market-data provider strategy derived from primary research, the alert-rule engine and AI-agent designs, the data model, the API contract, and the phased delivery plan. The frontend is being built first; the backend contract defined here is what the frontend is coded against from day one.

2. System Overview and Architecture

2.1 Product context

Meridian is a personal market intelligence system that behaves like an always-on financial research assistant. It combines near-real-time market data, portfolio context, user preferences, configurable monitoring rules, news and IPO information, deterministic calculations, and an LLM-based agent interface. Its defining feature is persistent personalization: the system knows what the user owns, watches, and which thresholds matter. It supports reactive conversations ("analyze Tata Steel") and proactive behavior ("tell me when Tata Steel reaches the 200-day average").

2.2 Architecture at a glance

Component	Technology	Responsibility
Web frontend	Next.js 16 (App Router), React 19, Tailwind 4	All user-facing screens; module-driven structure; talks to backend REST API only
Backend API	Spring Boot 3.5 (Java 21), REST, springdoc-openapi	Authentication, profile, portfolio, watchlist, alerts CRUD, market proxies, agent chat
Rule engine	Spring scheduling + ShedLock, in-memory condition-tree evaluator	Evaluates active AlertRules against cached market snapshots on a fixed cadence
AI agent	Spring AI 1.x (ChatClient, tool calling), OpenAI-compatible LLM API	Natural-language understanding, explanation, portfolio/stock interpretation; never the source of truth
Data layer	PostgreSQL 16 (JPA + Flyway), Redis 7 (Spring Data Redis)	System of record; quote/news/IPO caching, rate-limit counters, scheduler locks
Integrations	Provider adapters (Yahoo/Dhan/EODHD, Google News RSS/Marketaux, Apify IPO)	External market data, news and IPO feeds behind internal provider interfaces
Notifications	In-app + email (SMTP) first; web push, Telegram later	Delivers triggered alerts and digests; full delivery history retained

Table 2-1: Core components and responsibilities

2.3 Architecture principles

- **Tool-first AI.** The LLM orchestrates; deterministic services compute. The agent calls backend tools for every current number and never invents data.
- **Deterministic core.** Portfolio metrics, rule evaluation and scanner logic are plain code, unit-testable and auditable; the AI only interprets results.
- **Provider abstraction.** Every external feed sits behind an internal interface (MarketDataProvider, NewsProvider, IpoProvider, NotificationChannel) so vendors are replaceable without schema changes.
- **Asset-level batching.** One quote fetch per symbol per cycle feeds all users' rules; never one external request per user rule.
- **User-scoped isolation.** Every row that carries user data is keyed by user_id and authorized at the service layer.
- **Decision-support posture.** No order placement in MVP; estimates are labelled; data freshness (timestamp + source) is always displayed.

3. Technology Stack and Key Decisions

Layer	Technology	Version	Rationale
Frontend	Next.js (App Router), React, TypeScript strict	16 / 19	Already delivered; module-driven src/ structure; SSR + static hybrid fits dashboard content
Frontend UI	Tailwind CSS, Recharts, lucide-react	4 / 3.x	Utility styling with design tokens; charts for price history and allocation
Backend runtime	Java (virtual threads enabled)	21 LTS	Mature ecosystem; virtual threads make blocking I/O to providers simple and scalable
Backend framework	Spring Boot (Web, Security, Validation, Actuator)	3.5.x	Industry-standard REST stack; first-class Spring AI and Spring Data integration
Persistence	PostgreSQL + Spring Data JPA + Flyway	16	Relational fits the entity model; JSONB for condition trees; migrations are code-reviewed
Cache / counters	Redis + Spring Data Redis	7	Quote cache, provider rate-limit tokens, idempotency keys, scheduler locks (ShedLock)
AI	Spring AI (ChatClient, @Tool, structured output)	1.x	Declarative tool calling and typed entity conversion; provider-agnostic LLM client
Resilience	Resilience4j (retry, rate limiter, circuit breaker, fallback)	2.x	Provider latency/limits are the main failure source; fallback must degrade gracefully
Docs / API	springdoc-openapi (Swagger UI)	2.x	The API contract in this TRD is published as OpenAPI and kept executable
Build / run	Maven, Docker Compose, GitHub Actions	-	One compose file boots postgres, redis, api, web for local and staging

Table 3-1: Technology decisions

Deliberate non-choices: no Kafka/RabbitMQ (scheduled polling plus transactional outbox covers MVP scale), no microservices (one modular monolith with strict package boundaries), no ORM magic beyond JPA (hot paths use explicit queries), and no client-side market data (all provider calls are server-side to protect keys and control rate limits).

4. Market Data Research and Provider Strategy

Primary research (September 2026) into Indian-market data sources drives the provider strategy. The headline finding: **global freemium APIs serve NSE/BSE data only delayed or end-of-day**; true real-time feeds come from broker APIs (free to cheap for personal use) or licensed vendors.

4.1 Provider landscape

Provider	Coverage	Latency	Cost	Assessment
Yahoo Finance chart API (unofficial)	NSE/BSE quotes + 1-10y daily history	~15 min delayed	Free	Current default. No key, generous limits, but ToS grey area and no SLA; must cache aggressively and label freshness
Alpha Vantage	NSE EOD + fundamentals	EOD	Free 25 req/day	Useful fundamentals fallback; rate limit too low for quotes
Twelve Data	NSE quotes + history, WebSocket	Delayed / EOD	Freemium	Clean API; India real-time not guaranteed; viable paid alternative
EODHD	NSE/BSE EOD + fundamentals + splits	EOD	From ~USD 20/mo	Best structured EOD + fundamentals for Phase 2 research features
Finnhub	Primarily US	-	Freemium	Skipped: weak India coverage
NSE official site	Full NSE	Real-time	Free (scraping)	No public API; scraping is ToS-restricted and brittle; excluded
DhanHQ (broker API)	NSE/BSE REST + WebSocket + historical ticks	Real-time	Free for personal use	Best Phase 2 real-time feed: free market feed, no subscription for API access
Angel One SmartAPI	NSE/BSE REST + WebSocket	Real-time	Free with account	Strong free alternative; used as second broker adapter
Upstox API	NSE/BSE REST + WebSocket	Real-time	~INR 150+/yr from year 2	Cheap alternative; keep behind the same adapter
Zerodha Kite Connect	Full trading + feed + historical	Real-time	~INR 500-2,000/mo per app	Most mature; relevant at V4 when broker sync / trading lands
TrueData / GlobalDataFeeds	Licensed real-time NSE/BSE	Real-time	Commercial	Only if the product scales beyond personal use

Table 4-1: Indian market data providers (researched September 2026)

4.2 News and IPO data

Source	Use	Notes
Google News RSS	News provider now (frontend MVP already uses it)	Free, no key; company/topic queries with when:7d filter; relevance scoring happens in-app
Marketaux	News provider Phase 2	Entity/ticker tagging across 80+ markets including NSE; free monthly quota; best fit for per-company relevance
GNews / NewsAPI.org	Fallback search providers	100 req/day free tiers; NewsAPI free tier is dev-only with 24h delay; GNews supports country=in
Apify India IPO Tracker	IPO provider Phase 2	Clean JSON of upcoming IPOs, GMP and live QIB/NII/Retail subscription from BSE/NSE
Chittorgarh / InvestorGain	IPO subscription fallback	The de-facto sources for live subscription status; no official API, so access via the Apify actor or a maintained parser with caching

Table 4-2: News and IPO sources

4.3 Recommended strategy

- **Phase 1 (MVP):** MarketDataProvider backed by the Yahoo chart adapter with 60-second Redis caching per symbol, source + timestamp stored on every snapshot, and Resilience4j fallback to last-known-good quotes labelled STALE.
- **Phase 2 (real-time + fundamentals):** add a DhanHQ WebSocket adapter for live quotes of watchlist symbols (free, licensed feed via broker account), and EODHD for EOD history and fundamentals.
- **Phase 3 (V4, broker sync):** Kite Connect or SmartAPI adapters for read-only holdings sync; order workflows only with explicit confirmation, after legal review.
- **News:** Google News RSS now; Marketaux adapter behind the same NewsProvider interface when entity tagging is needed for the relevance engine.
- **IPO:** Apify India IPO Tracker actor behind IpoProvider; every allotment estimate is labelled with its methodology, and subscription figures carry their source and fetch time.

All provider interfaces return a normalized internal model (the QuoteSnapshot / NewsArticle / IPO entities of section 8) and attach provider name + provider timestamp. Missing values are never fabricated: the UI renders "data unavailable" with the last-good timestamp.

5. Backend Architecture (Spring Boot)

5.1 Module structure

A modular monolith: one deployable with package boundaries enforced by ArchUnit tests. Domains mirror the frontend modules, which keeps the codebase mentally symmetric.

```
meridian-api/
+-- api/          # REST controllers, request/response DTOs, error handling
|   +-- auth, profile, portfolio, watchlist, alerts, market, agent, news, ipo
+-- core/
|   +-- auth/      # users, credentials, JWT issuance, refresh tokens
|   +-- portfolio/ # holdings, transactions, metrics calculator
|   +-- watchlist/ # watchlist items, symbol master
|   +-- market/    # MarketDataProvider port, quote snapshots, indicators
|   +-- alerts/     # AlertRule, conditionTree model, evaluator, cooldown, history
|   +-- notify/     # NotificationChannel port, in-app + email adapters, digests
|   +-- agent/      # Spring AI ChatClient, @Tool registry, conversations, audit
|   +-- research/   # stock analysis layers, portfolio analysis orchestration
|   +-- ipo/        # IpoProvider port, IPO master, subscription snapshots
|   +-- news/       # NewsProvider port, articles cache, relevance scoring
+-- integrations/  # yahoo, dhan, eodhd, googlenews, marketaux, apify, smtp
+-- config/        # security, redis, scheduling, spring-ai, resilience, openapi
+-- db/migration/  # Flyway V1__init.sql, V2__...
```

Listing 5-1: Backend package layout

5.2 Cross-cutting components

Concern	Mechanism	Notes
Authentication	Spring Security 6, JWT access (15 min) + refresh (30 d) tokens	Stateless; BCrypt password hashing; all app endpoints authenticated except /api/v1/auth/**
Authorization	Service-level user checks (owner-or-404)	Every repository query filters by principal user id; integration tests prove isolation
Caching	Redis: quotes 60 s, news 5 min, IPO 10 min; TTL per provider	Cache key = provider + symbol + interval; stale-while-error returns last good value flagged STALE
Scheduling	Spring @Scheduled + ShedLock (JDBC/Redis lock provider)	Safe single-node or multi-node; cadences per section 6.3
Resilience	Resilience4j per integration: retry (2x, jitter), rate limiter, circuit breaker, fallback	Provider outages degrade to cached/stale data, never to fabricated numbers
AI integration	Spring AI ChatClient + @Tool registry; OpenAI-compatible endpoint	Model and base URL are configuration; token usage recorded per request for cost metrics
Observability	Actuator + Micrometer + structured JSON logs; provider health indicators	Metrics: provider latency/error/limit, scheduler lag, AI tokens/cost, cache hit ratio
API docs	springdoc-openapi UI at /swagger-ui	Contract-first checking against section 9

Table 5-1: Cross-cutting components

6. Alert Rule Engine Design

6.1 Rule model

The differentiator is a generic rule model instead of a fixed alert catalog. Every rule carries a JSONB conditionTree so nested AND/OR conditions of any family (price, percentage, volume, portfolio weight, fundamental, news, periodic) are representable without schema migrations.

```
AlertRule {
  id, userId, assetType, symbol,
  conditionTree: {                                // JSONB
    type: "AND",
    children: [
      { type: "PRICE", symbol: "TATASTEEL.NS", operator: ">=", value: 200 },
      { type: "NEWS", symbol: "TATASTEEL.NS", eventType: "RESULTS", sentiment: "NEGATIVE" }
    ]
  },
  triggerMode: ONE_TIME | RECURRING | PERSISTENT,
  cooldownMinutes, notificationChannels[], active,
  createdBy: USER | AGENT, lastTriggeredAt, createdAt
}
```

Listing 6-1: Alert rule model

6.2 Evaluation pipeline

- Scheduler tick: load distinct symbols referenced by active rules (asset-level batching).
- Fetch quotes cache-first (Redis 60 s TTL; miss goes to the provider through Resilience4j).
- Evaluate every matching rule's conditionTree in memory against the snapshot - a pure function, unit-tested per rule family.
- For true rules: apply cooldown (rule.lastTriggeredAt + cooldownMinutes) and duplicate suppression (unique rule_id + cycle key).
- Persist the trigger to alert history, create a Notification row per configured channel, and hand off to the delivery dispatcher.
- ONE_TIME rules deactivate after firing; RECURRING rules re-arm on their schedule; PERSISTENT rules stay armed until disabled.

6.3 Background job cadences

Job	Cadence	Purpose
Price rule monitor	Every 60 s within market hours; 5 min off-hours	Evaluate price / percentage / volume triggers
News monitor	Every 10 min	Detect new relevant company / sector / market events
Portfolio monitor	Hourly (user-configurable)	Concentration, allocation limits, major movements
IPO monitor	Every 30 min during bidding windows	Opening/closing, subscription milestones, listing
Daily summary	Once daily after close	Personalized end-of-day report (deterministic template first, AI summary in V2)
Weekly review	Weekly	Portfolio, thesis changes, watchlist events

Table 6-1: Scheduled jobs (PRD section 10)

60 s Quote freshness inside market hours	1 Notification per cooldown window	0 Per-user-rule provider calls
--	--	--

7. AI Agent Architecture (Spring AI)

7.1 Orchestration flow

The agent is an orchestration layer, not the source of truth. A user message reaches the agent service, which selects and executes backend tools, receives structured results, and produces an explanation or a proposed action. Material actions (rule changes) are executed by deterministic services and are audited either way.

```
User message
-> Agent service (Spring AI ChatClient + conversation memory)
-> Tool selection (@Tool methods, JSON-schema described)
-> Tool execution against core services (market / portfolio / alerts / ipo / news)
-> Structured tool result
-> LLM interpretation (facts | calculations | interpretation | assumptions)
-> Response, or proposed action requiring user confirmation
```

Listing 7-1: Agent request flow

7.2 Tool registry (initial set)

Tool	Backed by	Returns
get_stock_quote(symbol)	market/QuoteService	QuoteSnapshot + indicators
get_stock_history(symbol, range)	market/HistoryService	OHLCV series
get_company_fundamentals(symbol)	research/FundamentalsService	PE, margins, ROE, debt (provider-dependent)
get_latest_news(symbol or topic)	news/NewsService	Relevance-scored articles
get_portfolio / get_portfolio_metrics / get_holding	portfolio/PortfolioService	Positions, value, P&L, allocation, concentration
get_watchlist	watchlist/WatchlistService	Watchlist items
create_alert / update_alert / delete_alert / get_active_alerts	alerts/AlertService	AlertRule mutations + list (audited, createdBy=AGENT)
get_upcoming_ipos / get_ipo_details / get_ipo_subscription	ipo/IpoService	IPO master + subscription snapshots
analyze_stock(symbol) / analyze_portfolio()	research/AnalysisService	Layered evidence summary (fundamentals, valuation, technicals, news, portfolio fit)

Table 7-1: Agent tools

7.3 Guardrails and audit

- System prompt (Appendix A) enforces: use tools for current data; never invent data; separate facts, calculations, interpretations and assumptions; no guaranteed-return claims; say when data is stale.
- Structured intents: free-text like "Tata Steel 200 par aaye tyare alert karje" is converted with .entity(CreateAlertIntent.class) into a typed payload, validated server-side, then persisted by the normal AlertService.
- Every tool execution writes an AgentAction row (tool, input, result digest, conversation, confirmation status) - AC-12 auditability.
- Token usage and latency are recorded per request for cost monitoring; a per-user daily token budget guards runaway cost.
- Agent-created rules are visible and editable in the Alerts UI (UI remains the source of truth for configuration).

8. Data Model and Persistence

Entity	Key fields	Notes
users	id, email, password_hash, name, timezone, currency	Account identity; auth via Spring Security
user_preferences	user_id, risk_level, horizon, budget, allocation_limits jsonb, sector_prefs	Personalization inputs for the agent
holdings	user_id, symbol, quantity, avg_buy_price, thesis	Current portfolio; transactions table lands in V4

Entity	Key fields	Notes
watchlist_items	user_id, symbol, type	Owned or unowned monitoring
alert_rules	user_id, asset_type, symbol, condition_tree jsonb, trigger_mode, cooldown_minutes, channels[], active, created_by, last_triggered_at	Section 6 model
alert_events	rule_id, triggered_at, snapshot jsonb	Trigger history; powers the alerts timeline
notifications	user_id, event_id, rule_id, title, body, severity, channel, status, sent_at	Full delivery history
instruments	symbol, exchange, name, sector	Symbol master seeded per user universe
quote_snapshots	symbol, price, volume, provider, provider_time, fetched_at	Market observations (also the stale-fallback source)
news_articles	provider_id, symbol_or_topic, published_at, relevance	News cache with dedupe on provider_id
ipos	company, open/close dates, price band, lot_size, issue_size	IPO master
ipo_subscription_snapshots	ipo_id, timestamp, qib/nii/retail	Subscription history
conversations / messages	user_id, title / conversation_id, role, content, tool_meta jsonb	Agent memory and history
investment_theses	user_id, symbol, thesis, assumptions, review_at	Long-term memory (V3)
agent_actions	conversation_id, tool, input jsonb, result_digest, confirmed, status	Auditability (AC-12)

Table 8-1: Entity model (PostgreSQL, Flyway-managed)

- condition_tree and allocation_limits are JSONB - new rule families need no migration.
- Every user-scoped table carries user_id with composite indexes (user_id, symbol) for the hot paths.
- Flyway migrations are the only schema path; V1 creates auth + portfolio + alerts MVP tables.

9. API Contract

9.1 Conventions

- Base path /api/v1; JSON bodies; camelCase fields; ISO-8601 timestamps with offset.
- Auth: Authorization: Bearer on every endpoint except /api/v1/auth/**.
- Errors: RFC 7807 application/problem+json with type, title, status, detail, traceId.
- Lists are cursor-paginated (?cursor=, ?limit=, max 100); market queries accept symbol lists capped at 28.
- All endpoints are user-scoped server-side; ids are UUIDs.

9.2 Endpoint inventory

Method	Endpoint	Purpose	Auth
POST	/api/v1/auth/register	Create account	Public
POST	/api/v1/auth/login	Authenticate, issue tokens	Public
POST	/api/v1/auth/refresh	Rotate access token	Refresh token
GET	/api/v1/profile	Get user profile	Bearer
PUT	/api/v1/profile/preferences	Update preferences (risk, horizon, limits)	Bearer
GET	/api/v1/portfolio	Portfolio with computed metrics	Bearer
POST / PUT / DELETE	/api/v1/portfolio/holdings/{id}	Manage holdings	Bearer
GET / POST	/api/v1/watchlist	List / add watch items	Bearer
DELETE	/api/v1/watchlist/{symbol}	Remove watch item	Bearer
GET / POST	/api/v1/alerts	List rules / create rule	Bearer
PUT / DELETE	/api/v1/alerts/{id}	Edit / delete rule	Bearer
GET	/api/v1/alerts/events	Trigger history	Bearer
GET	/api/v1/notifications	Notification list + unread count	Bearer
POST	/api/v1/agent/chat	Chat with PA (streams SSE)	Bearer
GET	/api/v1/agent/conversations/{id}	Conversation history	Bearer
GET	/api/v1/stocks/{symbol}	Quote + indicators + freshness	Bearer
GET	/api/v1/stocks/{symbol}/analysis	Layered stock analysis	Bearer
GET / POST / DELETE	/api/v1/news[?symbol=]	Relevance-filtered news	Bearer
GET	/api/v1/ipos, /api/v1/ipos/{id}, /{id}/subscription	IPO list, details, subscription	Bearer

Table 9-1: API surface (v1)

9.3 Example payloads

```
POST /api/v1/alerts
{ "assetType": "EQUITY", "symbol": "TATASTEEL.NS",
  "conditionTree": { "type": "PRICE", "operator": ">=", "value": 200 },
  "triggerMode": "ONE_TIME", "cooldownMinutes": 0,
  "notificationChannels": ["IN_APP", "EMAIL"] }

201 Created
{ "id": "9f1c...", "symbol": "TATASTEEL.NS", "active": true,
  "createdBy": "USER", "createdAt": "2026-09-26T10:12:00+05:30" }

409 Conflict (problem+json)
{ "type": "https://meridian.app/problems/symbol-unknown",
  "title": "Unknown symbol", "status": 409,
  "detail": "TATASTEEL.BOGO does not match any instrument", "traceId": "..." }
```

Listing 9-1: Create-alert contract

10. Security and Compliance

Area	Measure
Authentication	Spring Security 6 + JWT; BCrypt (strength 12); refresh-token rotation with reuse detection
Session / token handling	Short-lived access tokens; refresh tokens stored hashed; revocation list in Redis
Transport	HTTPS everywhere (HSTS); no provider keys in the browser, ever
Data isolation	User-scoped queries enforced in repositories; authorization tests per endpoint (AC-14)
At rest	PostgreSQL volume encryption at the platform layer; sensitive columns (token hashes) hashed
Audit	AgentAction + alert_events + notification delivery records; append-only
Secrets	Environment/secret-store only (no secrets in git); .env excluded by .gitignore; rotation runbook
AI safety	Tool allow-list, output schema validation, no LLM write access beyond the alert rule services
Financial safety	Decision-support positioning; explicit uncertainty on estimates; no order placement in MVP
Provider licensing	Respect provider ToS; per-provider attribution and freshness labels in the UI

Table 10-1: Security controls

11. Frontend Architecture

11.1 Current module map (delivered)

The codebase was restructured into a module-driven src/ layout. Each feature module owns its components, hooks and logic and exposes a public index.ts; shared UI primitives and formatting live in components/ and lib/; domain types are split per area in types/. API route handlers (proxying Yahoo/Google News) keep the current local mode working without a backend.

Module	Owns	Screen
modules/market	MarketProvider store, InstrumentPanel, NewsList, PriceChart, TradingViewWidget, yahoo/news adapters, indicators, seed	(core)
modules/dashboard	CommandView - portfolio health, alerts, news, scanner digest	/ (dashboard)
modules/markets	MarketsView - watchlist grid	/markets
modules/portfolio	PortfolioView, calculatePortfolio	/portfolio
modules/alerts	AlertsView, AlertList, alert logic	/alerts
modules/scanner	ScannerView, useScanner, scanner engine	/scanner
modules/research	ResearchView	/research
components / lib / types	Shell, ui primitives, format utils, per-domain types	(shared)

Table 11-1: Frontend modules today

11.2 Backend integration strategy (this release)

A typed API-client layer is being added now so the Spring Boot backend can be plugged in later without touching screens. An environment switch selects the data source; both adapters implement the same TypeScript contracts.

Setting	Values	Behaviour
NEXT_PUBLIC_API_MODE	local (default) rest	local = current browser-local store + Next route proxies; rest = Spring Boot backend
NEXT_PUBLIC_API_BASE_URL	e.g. http://localhost:8080	Only used in rest mode; token attached from the auth store
Mock auth	local mode	AuthProvider auto-issues a demo session so existing screens keep working; login/register screens exercise the real flow

Table 11-2: Integration switches

11.3 Screen inventory and status

Screen	PRD ref	Status
Dashboard (portfolio health, movement, warnings, alerts)	18	Done (local mode)
Markets / watchlist	18	Done (local mode)
Portfolio (holdings, allocation, P&L;)	18	Done (local mode)
Alerts (rules, history, create/edit)	18	Done (local mode)
Scanner (templates, results, CSV export)	12/extension	Done (local mode)
Research (news + instrument context)	18	Done (local mode)
Settings (profile, risk, notification channels)	18	In progress (this release)
Auth (login / register)	7.1	In progress (this release, mock session)
AI Chat (streaming, citations)	18	V2 (needs backend agent)
Stock detail page (fundamentals, AI analysis)	18	V2 (needs fundamentals data)
IPO page (issue details, subscription, fit)	18	V2 (needs IPO provider)

Table 11-3: Screen status

- State: the MarketProvider context remains the single store in local mode; in rest mode the same context hydrates from the REST API and mutates via API calls, so screens do not change.
- Design system: existing Tailwind tokens and ui.tsx primitives are the component library; new screens must compose them.
- Freshness: every price surface shows timestamp + source, in line with AC-13 (stale shown, never fabricated).

12. Non-Functional Requirements

< 1 s Authenticated API target (excl. provider latency)	99% Alert cycle completion despite provider faults	100% AI tool calls audited
--	--	--------------------------------------

Area	Requirement
Availability	Dashboard and alert processing resilient to transient provider failures (cache fallback, STALE labelling)
Performance	Sub-second server processing for authenticated APIs excluding external provider latency; p95 endpoint budget tracked in CI
Scalability	Market data cached/shared by asset, never duplicated per user rule; provider calls batched
Reliability	Alert evaluation idempotent; exactly one notification per cooldown window (AC-05)
Auditability	Every AI tool call and material agent action traceable (AC-12)
Observability	Structured logs, metrics, traces, provider health checks, scheduler lag and queue depth
Data freshness	UI shows timestamp + source for live or cached data
Extensibility	Provider interfaces for market data, news, notifications and the LLM are replaceable

Table 12-1: NFR targets (PRD section 20)

13. Environments and DevOps

Environment	Purpose	Composition
Local	Development	docker-compose: postgres, redis, api (Spring Boot), web (Next.js); seeded demo data
Staging	Integration + demo	Single VM or PaaS (Render/Railway): managed Postgres, Redis, api, web; provider sandbox keys
Production	Personal deployment	PaaS or small VPS; backups nightly; structured logs shipped off-box

Table 13-1: Environments

- CI (GitHub Actions): frontend - lint, tsc, build; backend - Maven test, ArchUnit boundaries, Flyway checksum validation, Testcontainers integration tests (Postgres + Redis).
- Migrations run as a release step (Flyway validate + migrate); backward-compatible changes only while frontend deploys independently.
- Config via environment: DATABASE_URL, REDIS_URL, JWT_SECRET, LLM_API_KEY + model, provider keys; .env.example documented in the repo.

14. Delivery Plan and Current Status

Phase	Deliverables	Definition of done	Status
Foundation (frontend)	Next.js app, module structure, dashboard/markets/portfolio/alerts/research/scanner screens, design system	All P0 screens render against local mode; build green	Done
Frontend readiness (this release)	Typed API client + contracts, auth module (mock session), settings module, integration switches	Screens compile against v1 contracts; mock/live switch in place	In progress
Foundation (backend)	Spring Boot skeleton, Postgres + Flyway, auth + profile, docker-compose, CI	User can register and persist core profile data (PRD 25)	Next
Portfolio + watchlist	Holdings/watchlist APIs, metrics calculator, dashboard hydration	Dashboard shows persisted holdings and computed metrics	Planned
Market layer	Provider abstraction, Yahoo adapter, Redis cache, quote endpoints	Stock data displays timestamped quotes with STALE fallback	Planned
Alert engine	Rule model, scheduler, evaluator, history	Tata Steel ≥ 200 use case works end-to-end without the browser open	Planned
Notifications	In-app + email adapters, delivery history	Triggered alert is delivered and recorded (AC-05)	Planned
AI layer	Spring AI integration, tool registry, chat UI	User can create an alert through natural language (AC-03)	Planned
Intelligence	Stock/portfolio/news/IPO analysis services	AI explains portfolio and researched assets with tool data	Planned
Autonomy (V3)	Composite rules, thesis memory, digests, relevance engine	Agent proactively surfaces relevant events with low noise	Planned

Table 14-1: Phased delivery plan mapped from PRD section 25

15. Risks and Mitigations

Risk	Impact	Mitigation
Yahoo Finance is unofficial and can block or change without notice	High	Provider abstraction + Resilience4j fallback to last-good STALE data; Phase 2 moves live quotes to DhanHQ/SmartAPI licensed broker feeds
LLM hallucination	High	Tool-first architecture, structured outputs, deterministic calculations, source context, guardrail system prompt
Market data latency/inaccuracy	High	Timestamp + source on every observation, freshness labels, cache rules, provider health checks
Notification overload	Medium	Relevance scoring, cooldowns, deduplication, user-configurable severity
Vendor lock-in	Medium	Provider interfaces with normalized internal models; adapters are thin
API rate limits / cost	Medium	Asset-level batching, 60 s cache, scheduled polling, provider-aware backoff, AI token budget
Financial / regulatory exposure	High	Decision-support positioning, transparent uncertainty, explicit confirmations, legal review before broker/trade features
Security breach	High	Least privilege, secure auth, secret management, audit logging, encrypted storage
Single-maintainer scope creep	Medium	Modular monolith with enforced boundaries; phases gated by PRD acceptance criteria

Table 15-1: Risk register

Appendix A. Agent System Prompt (baseline)

You are a personal investment decision-support agent. Core rules: (1) Use tools for current market, IPO and portfolio data. (2) Never invent missing data. (3) Distinguish facts, calculations, interpretations and assumptions. (4) Consider the user profile and existing portfolio when relevant. (5) Explain risks and uncertainty. (6) Do not claim guaranteed returns, guaranteed allotment or certainty about future prices. (7) For material account or broker actions, require explicit confirmation. (8) Keep alert creation and rule evaluation deterministic in the backend. (9) When data is stale or unavailable, say so. (10) Prefer a useful, concise response with actionable next steps.

Appendix B. References

- Personal AI Investment Agent PRD v1.0 (September 2026) - source product document.
- Spring AI reference (tool calling, structured output): docs.spring.io/spring-ai.
- Market data: query1.finance.yahoo.com chart API (unofficial); eodhd.com; twelvedata.com; alphavantage.co.
- Broker feeds: dhan.co (DhanHQ API), smartapi.angelone.in, upstox.com developer docs, kite.trade.
- News: news.google.com/rss (in use), marketaux.com (entity-tagged), gnews.io.
- IPO data: apify.com/data_daemon/india-ipo-tracker (upcoming IPOs, GMP, subscription); chittorgarh.com live status.